

Auto-Generated FAQ from Medical Chatbot Logs

Specialties: Radiology, Physiotherapy, Cardiovascular

	1. Abdul Ghani (F23607005) 2. Anas Bhatti (F23607044) 3. Muhammad Salman (F23607037) 4. [Member 4 - name and roll number TBD] 5. [Member 5 - name and roll number TBD]
--	--

Project Type	Semester Project, Healthcare NLP
Team Size	5 members
Stack	Next.js, FastAPI, BGE-M3, HDBSCAN, Qdrant, MongoDB
Deliverable	Web app with 3 chatbots + auto-FAQ page + admin dashboard
Weightage	Proposal and approval contribute 20% of course marks

1. Problem Statement and Motivation

Hospitals, diagnostic centers, and rehab clinics receive the same patient questions every single day. Front-desk staff and junior doctors keep explaining scan preparation, exercise routines, recovery timelines, and medication usage again and again. The conversations are stored somewhere (web chat, WhatsApp, or paper notes) but almost never mined, so help sections on clinic websites stay outdated, response times stay slow, and new symptom trends are spotted late.

There is no straightforward system that turns this raw chat history into a clean, ranked, continuously updated knowledge base. Existing tools either need manual FAQ writing or are tied to one platform like Intercom. We want to build something that works directly off a clinic's own web chatbot logs, covers three medical specialties (Radiology, Physiotherapy, Cardiovascular), and handles the kind of Urdu and English code-switched text that local patients actually write.

2. Project Objectives

- Build a web application that hosts three specialty chatbots and logs every conversation.
- Run an NLP pipeline that clusters semantically similar patient questions across all logged chats.
- Auto-generate a ranked FAQ page per specialty from those clusters.
- Detect emerging issues (clusters that grow fast over a short time window) and alert admins.
- Provide an admin dashboard so clinicians can review, edit, or reject auto-generated FAQs before they go public.
- Evaluate the system with quantitative clustering metrics and qualitative clinician ratings.

3. Project Tasks and Scope

The work is grouped into eight tracks. Each track has a clear input, output, and owner (see Section 12). Tracks run partly in parallel to fit a 16-week semester.

Task	Description
T1. Web frontend	Next.js site, three specialty chat widgets, public FAQ page, admin dashboard.
T2. Backend and APIs	FastAPI services, MongoDB schema, chat logger, optional WhatsApp or email connectors.
T3. Conversation processing	Text segmentation, question identification, PHI scrubbing, intent normalisation.
T4. Embedding and clustering	Generate embeddings (BGE-M3), HDBSCAN plus BERTopic, cross-channel deduplication.

T5. Answer selection and polish	English best-answer pick per cluster (no generation), rule-based output polish.
T6. Emerging issue detection	Temporal embedding analysis, anomaly detection, in-dashboard alerts.
T7. Admin dashboard	FAQ approval / edit / reject workflow, analytics, manual override, CSV export.
T8. Evaluation and reporting	Clustering metrics, clinician ratings, latency tests, final report, demo video.

4. Functional Requirements

ID	Functional Requirement
FR1	User can select one of three medical specialties on the landing page.
FR2	User can chat with a specialty-specific bot in real time.
FR3	Every chat turn is stored in MongoDB with session id, specialty tag, and timestamp.
FR4	A background pipeline clusters past conversations on a schedule (default hourly).
FR5	Each specialty has a public FAQ page that lists ranked Q and A entries.
FR6	Admins can log in and approve, edit, reject, or pin candidate FAQs.
FR7	System detects fast-growing clusters in a 24-hour window and raises an alert.
FR8	Admins can write a manual FAQ that overrides any auto-generated entry.
FR9	Admins can export all clusters and FAQs as CSV for offline review.

5. Non-Functional Requirements

ID	Non-Functional Requirement
NFR1	Chatbot reply latency under 5 seconds for typical input (under 200 tokens).
NFR2	Full pipeline runs end to end on 10k conversations in under 30 minutes on a 4-core 8GB VM.
NFR3	PHI scrubbing (names, phone numbers, IDs) applied before any conversation is stored.
NFR4	Entire stack runs from one docker-compose file on a Linux VM.
NFR5	Frontend renders correctly on mobile screens since most patients use phones.
NFR6	Multilingual embeddings (BGE-M3) so Urdu and English code-switched text clusters correctly.
NFR7	Code is version-controlled in Git with unit tests for parsers and clustering helpers.
NFR8	If the pipeline is down, chatbot keeps working and FAQ page shows the last good snapshot.

6. Datasets

We mix public medical Q and A datasets, a couple of general dialogue corpora for turn-segmentation training, template-built synthetic conversations (no LLM, just parameterised patterns), and a small set of anonymised logs from one or two local partner clinics if consent can be arranged.

Dataset	Size	Use in Project
MedDialog (English)	approx. 250k dialogues	Training and evaluation of question clustering.
HealthCareMagic-100k	100k Q/A pairs	Bootstrap answer pool for the retrieval index and clustering evaluation.
iCliniq-10k	10k Q/A pairs	Held-out evaluation set, especially cardiovascular.
MTSamples	approx. 5k clinical notes	Medical vocabulary and abbreviation reference for normalisation.
Customer Support on Twitter	approx. 2.8M tweets	General support dialogue structure for the segmenter.
Ubuntu Dialogue Corpus	approx. 1M dialogues	Conversation turn segmentation training.
Template-built specialty Q/A	approx. 3k per specialty	Cold-start corpus built from parameterised templates (no LLM).
Partner clinic anonymised logs	approx. 1k each (target)	Pilot validation and Urdu / English code-switched samples.

7. Web Application Overview

The product is a single web application with three working surfaces. (1) A public landing page lets a patient pick a specialty and chat with the matching bot. The bot is fully retrieval-based: it embeds the user's question and returns the closest stored answer from a curated FAQ index. No generative model is used. The index is bootstrapped from public medical Q and A datasets and then enriched over time by our clustering pipeline. Every turn is stored in MongoDB along with session metadata. (2) A public FAQ page for each specialty lists the auto-built questions and answers, refreshed as new clusters form. (3) An admin dashboard lets clinicians approve, edit, or reject candidate FAQs before they go public, and review trending issues.

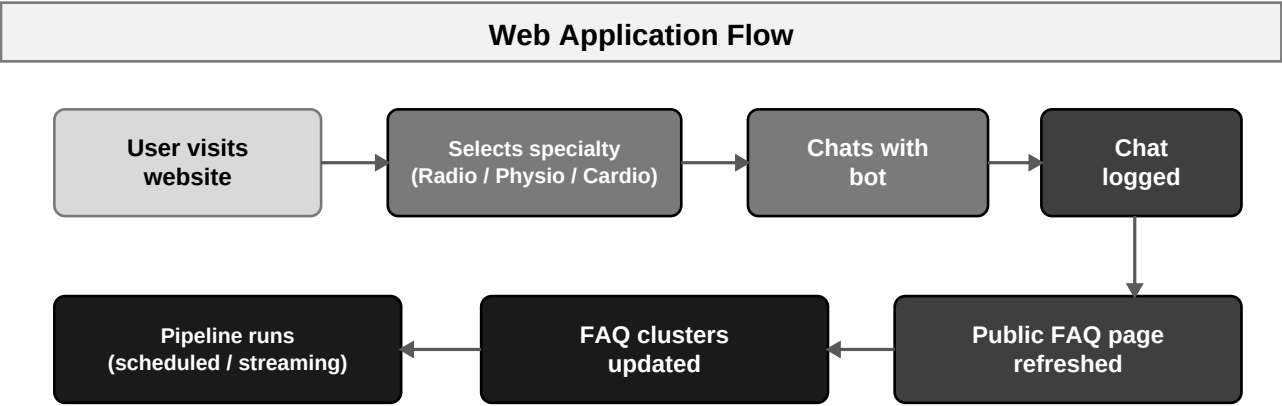


Fig 1. End-to-end user flow from chat to published FAQ.

8. System Architecture

The system has five layers. Users hit a Next.js frontend that serves both the chatbot widgets and the public FAQ page. A FastAPI gateway routes traffic to two core services: the retrieval service (semantic search over the FAQ index, no LLM) and the FAQ service (clustering pipeline). MongoDB stores chat sessions, Qdrant holds embeddings, and a separate FAQ store keeps the curated knowledge base plus analytics. Each layer is independently deployable from one docker-compose file.

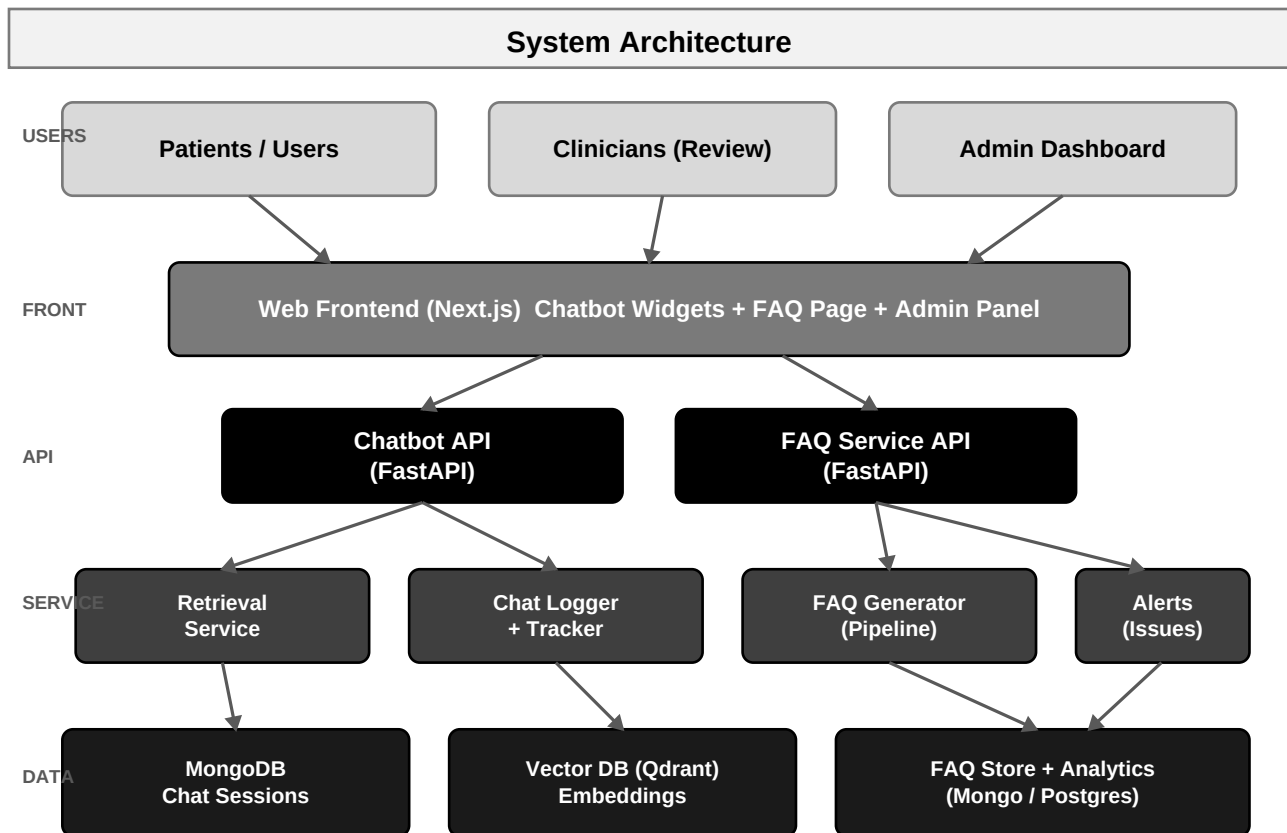
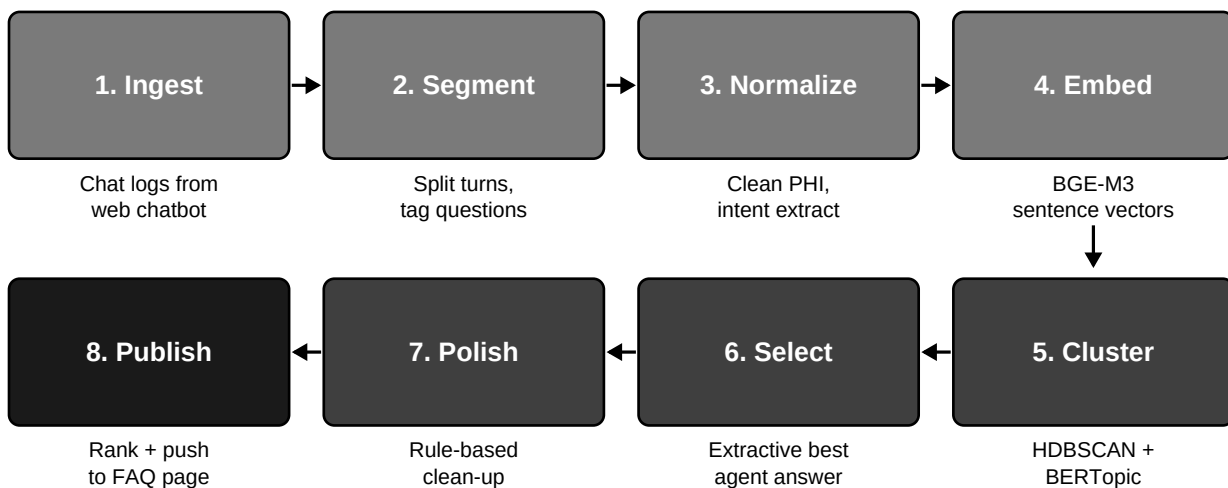


Fig 2. Five-layer architecture of the chatbot plus FAQ generator.

9. Tentative Pipeline

Logged chats flow through eight stages. Stages 1 to 4 prepare and embed the data, stages 5 to 7 cluster and synthesise, and stage 8 publishes. The pipeline runs on a schedule (default hourly via Celery beat) and can be triggered on demand from the admin dashboard.

FAQ Generation Pipeline



Continuous loop: new chats feed back to step 1

Fig 3. Eight-stage FAQ generation pipeline.

Step-by-step details

Step 1: Ingest. Pull new chat sessions from MongoDB. Each session has a turn list and a specialty tag. Optionally also load partner-clinic export files.

Step 2: Segment. Split each session into single turns and classify each user turn as question, follow-up, or non-question. Implemented with a small fine-tuned classifier.

Step 3: Normalise. Strip names, phone numbers, IDs, and dates (PHI). Lower-case, fix typos with ftfy, language-tag using langdetect.

Step 4: Embed. Encode each candidate question with BGE-M3 (multilingual). Store the embedding plus metadata in Qdrant.

Step 5: Cluster. Run HDBSCAN on the embeddings to get density-based clusters. BERTopic gives topic labels for each cluster.

Step 6: Select. For each cluster, score the agent responses by similarity to the cluster centroid, length, and recency. Pick the highest-scoring existing reply as the canonical answer. This is fully extractive, with no LLM generation.

Step 7: Polish. Apply rule-based clean-up: normalise whitespace, fix punctuation, trim to a maximum sentence count, and prepend a configurable greeting and closing.

Step 8: Publish. Rank clusters by frequency, recency, and severity. Push the top N to the public FAQ page. Mark sudden-growth clusters as emerging issues.

10. NLP Techniques Used

- Conversation segmentation with turn-level question identification using a small fine-tuned classifier.
- Intent extraction and question normalisation (paraphrase resolution) using rule-based plus embedding similarity.
- Semantic retrieval for the chatbot: dense nearest-neighbour search over the FAQ index (no LLM in the loop).
- Semantic clustering with multilingual sentence embeddings (BGE-M3, E5-multilingual).
- Density-based clustering with HDBSCAN; topic labels via BERTopic.
- Extractive answer selection per cluster: rank existing agent replies by centroid similarity, length, and recency. Pick the top reply as the canonical answer.
- Rule-based output polishing: whitespace, punctuation, length trimming, greeting and closing templates.
- Temporal embedding analysis for emerging-issue detection (rising clusters over a sliding window).
- Cross-channel deduplication (same issue logged across web chat and WhatsApp).
- PHI scrubbing with regex plus a small NER model before any storage of training-eligible text.

11. Libraries, Tools, and IDE

Category	Choice
IDE	VS Code (primary), Jupyter for ML experiments, DataGrip for Mongo browsing
Languages	Python 3.11 for backend and ML, TypeScript for frontend
Frontend	Next.js 15, React 19, Tailwind CSS, shadcn/ui, Zustand
Backend	FastAPI, Uvicorn, Pydantic v2
Background jobs	Celery + Redis broker, Celery beat for scheduling
Databases	MongoDB (chat sessions, FAQ store), Postgres (analytics, optional)
Vector DB	Qdrant in Docker, Weaviate as a fallback
Embeddings	sentence-transformers, BAAI/bge-m3, intfloat/e5-multilingual
Clustering	scikit-learn (KMeans baseline), HDBSCAN, BERTopic, UMAP for visualisation
Retrieval / IR	FAISS or Qdrant for dense kNN, rank_bm25 as a sparse baseline
Extractive selection	Custom Python scorer (centroid similarity + length + recency), MMR for diversity
Text polish	Pure Python rule-based cleaners (regex, NLTK sentence splitter, custom templates)
NLP utilities	spaCy (English), Indic-NLP, ftfy, langdetect, regex
Evaluation	Hugging Face evaluate, custom Python scripts, scikit-learn metrics
Versioning	Git + GitHub, conventional commits, pull requests
Containers	Docker, Docker Compose
Hosting	VPS (Hetzner or DigitalOcean), NGINX reverse proxy, systemd
Project management	GitHub Projects (Kanban), Notion for notes, Discord for daily sync

12. Task Split (5 Members)

Eight project tracks are bundled into five ownership areas, one per member. Members meet at the boundaries (API contracts, data schemas, evaluation hooks). Each row below maps one member to one focused area.

#	Member	Focus Area	Concrete Responsibilities
1	Abdul Ghani (F23607005)	Frontend and UI	Next.js site, three specialty chat widgets, FAQ page layout, admin dashboard UI.
2	Anas Bhatti (F23607044)	Conversation Processing	Turn segmentation, question identification, intent normalisation, PHI scrubbing.
3	Muhammad Salman (F23607037)	Embedding and Clustering	Embedding pipeline (BGE-M3), HDBSCAN with BERTopic, cross-channel deduplication.
4	[Member 4] ([Roll No 4])	Backend, APIs, Data	FastAPI services, MongoDB schema, chat logger, optional WhatsApp / email ingestion.
5	[Member 5] ([Roll No 5])	Selection, Issues, Eval	Extractive answer selection, rule-based polishing, emerging-issue detection, evaluation.

13. Expected Results

- Live demo URL hosting three specialty chatbots and three FAQ pages.
- At least 5,000 synthetic conversations processed end to end, plus 1,000 or more real conversations where consent allows.
- FAQ page with at least 30 high-quality ranked Q and A entries per specialty by demo day.
- Clustering quality: silhouette score above 0.45 on the test split and manual cluster purity above 80% on a 100-cluster sample.
- Answer quality: at least 70% of generated answers rated useful as-is by a clinician on a 50-FAQ sample.
- Pipeline runtime: full run on 10k chats under 30 minutes on a 4-core 8GB VM.
- Emerging-issue detection that surfaces at least three real trending clusters during a two-week pilot, with false-positive rate under 30%.
- Final project report (12 to 20 pages), recorded demo video, and open-source repository.

14. Value Addition by Our Work

- LLM-free design. The whole system uses classical NLP plus sentence embeddings only. Every answer the user sees is a real, human-written agent reply selected by our pipeline, never a generated one. This avoids hallucination risk, which is critical in a medical setting, and keeps every output traceable.
- Cross-specialty pipeline. Most existing FAQ tools target a single domain. Our pipeline handles three specialties from one codebase, so it generalises.
- Code-switched language support. Patients in Pakistan often mix Urdu and English. We pick BGE-M3 (multilingual) so this is supported at the embedding layer.
- Continuous and incremental. The FAQ regenerates as new chats arrive, instead of being a one-shot batch job.
- Clinician-in-the-loop. We deliberately design a review-and-approve workflow rather than a black-box auto-publish, since medical content needs a human check.
- Emerging-issue detection. Beyond static FAQs, our system flags rising clusters as early-warning signals, which existing FAQ tools do not do.
- Local hosting friendly. The full stack runs on a single 4-core 8GB VM, so small clinics and university labs can self-host without cloud costs.
- Open dataset potential. With partner consent, we can release a clustered anonymised medical FAQ dataset for follow-up research.

15. Evaluation Plan

Evaluation uses both intrinsic (cluster quality) and extrinsic (FAQ usefulness) metrics, plus a small user study if a partner clinic agrees.

- Clustering quality: silhouette coefficient, Davies-Bouldin index, and manual cluster purity on a 100-cluster sample.
- Question normalisation: pairwise accuracy on 500 hand-labelled paraphrase pairs.
- Answer quality: three graders rate 50 FAQ entries on factual correctness, tone, and completeness (5-point scale).
- Cross-specialty consistency: pipeline run separately on each specialty must give comparable cluster quality (silhouette within 0.05).
- Latency: chatbot response time and full pipeline run time measured under 4-core VM load.
- Emerging-issue detection: simulate a synthetic spike in one cluster and measure time-to-alert plus false-positive rate.
- User study (optional): ten patients try the bot and rate the experience on a short questionnaire.

16. Project Timeline (16-week semester)

Weeks	Phase	Key Deliverables
1-2	Proposal and approval	Approved proposal, GitHub repo, project skeleton, dataset download.
3-4	Frontend skeleton	Landing page, three chatbot widgets (placeholder replies), empty FAQ page.
4-6	Backend and logging	FastAPI service, MongoDB schema, chat logger, retrieval service returning real answers from
6-8	Data preprocessing	Dataset loaders, PHI scrub, conversation segmentation, normalisation.
8-10	Embedding and clustering	Pipeline running on 5k chats, first cluster output, BERTopic labels.
10-12	Selection and polish	Auto-built FAQs (extractive plus rule-based polish) appearing on the public FAQ page.
12-13	Emerging-issue detection	Trending-cluster alerts and admin notifications working in the dashboard.
13-14	Pilot data collection	Optional pilot with a partner clinic, real conversation ingest.
14-15	Evaluation	Compute metrics, clinician rating session, latency tests.
15-16	Report and demo	Final report, recorded demo video, presentation, GitHub release.

17. Academic Value

The project covers the classical NLP stack relevant to this semester: text preprocessing, sentence embeddings, dense retrieval, unsupervised clustering, extractive selection, rule-based output polishing, and temporal anomaly detection. Every component is implementable, measurable, and presentable with quantitative results. By choosing to avoid LLMs entirely, the work stays inside well-understood NLP territory and every output is traceable to a real human-written answer. The final deliverable is a working web system that can be demonstrated live and evaluated end to end. The healthcare focus also gives a clear narrative for the final report and leaves a clean extension path toward a follow-up paper or an open clustered medical FAQ dataset.

18. References (Tentative)

- [1] Chen Z. et al., MedDialog: Large-scale Medical Dialogue Datasets, EMNLP 2020.
- [2] Reimers N., Gurevych I., Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks, EMNLP 2019.
- [3] Xiao S. et al., BGE M3-Embedding: Multi-Lingual, Multi-Functional, Multi-Granular Text Embeddings, 2024.
- [4] McInnes L., Healy J., HDBSCAN: Hierarchical Density Based Clustering, JOSS 2017.
- [5] Grootendorst M., BERTopic: Neural Topic Modelling with Class-Based TF-IDF, 2022.
- [6] Erkan G., Radev D., LexRank: Graph-based Lexical Centrality as Saliency in Text Summarisation, JAIR 2004.
- [7] Robertson S., Zaragoza H., The Probabilistic Relevance Framework: BM25 and Beyond, FNTIR 2009.
- [8] Li J. et al., A Survey on Medical Question Answering Systems, ACM Computing Surveys 2023.